

CHAPTER-SIX

Analysis- Structuring Requirements

1 Overview

This handout covers some tools and techniques that can be used to structure the requirements gathered during the Analysis Phase of the SDLC. It describes Process Modelling and Data Modelling, and the structured analysis techniques used for each - Data Flow Diagram (DFD) and E-R (Entity Relationship) analysis, respectively. DFDs are looked at in detail whilst E-R analysis will be covered on a subsequent handout.

On Handout 7, it was stated that the **purpose** of the Analysis phase is as follows:

1. To determine how the current information system in the organization functions
2. To assess what users would like to see in the new system i.e. gather requirements
3. To structure the requirements so that they can be translated into technical system specifications.

This handout covers some tools and techniques that can be used for step 3 – to structure the requirements gathered. This is the second part of the process called 'Requirements Analysis'.

2 Structuring System Requirements

(Reference: Ref B, Chapter 5 – Structuring System Requirements: Process Modelling;
Chapter 6 – Structuring System Requirements: Conceptual Data Modelling.)

The information gathered during the Requirements Gathering process needs to be organized into a form that is a meaningful representation of the existing system and of the requirements for the new system.

This is done by producing *models* – of the processing elements and data transformations and that of the structure of the data.

This entire process is the *structuring of the requirements*.

These models can then be used as a starting point of the Design phase of the SDLC.

There are two stages in the structuring process:

- Process Modelling
- Conceptual Data Modelling.

Process Modelling involves graphically representing the processes, or actions, that capture, manipulate, store and distribute data between a system and its environment and between components with the system.

A **data flow diagram (DFD)** is a commonly used form of a process model, and is the technique we will deal with in this chapter.

Conceptual Data Modelling involves representing the data in a system or organization, to show the overall structure of the data. A data model should be independent of any DBMS and of other implementation considerations.

Entity-relationship (E-R) data models are commonly used diagrams that show how data is organised in a system.

Normally, Process Modelling and Data Modelling take place at the same time, with different teams working on each. The deliverables from each are shown in the table below.

Deliverables from the modelling activities		
Process Modelling	1. Context DFD	Shows the scope of the system, what elements are inside & outside the system
	2. DFDs of current physical system	Existing system - specify which people & technologies are used, in which processes to move/transform data, accepting inputs & producing outputs
	3. DFDs of new logical system	New system – technology independent (logical) diagrams to show data flow, structure and functional requirements
	4. Thorough descriptions of each DFD component	Every object included in all the above diagrams should be described in a <i>project dictionary</i>
Conceptual Data Modelling	1. E-R diagram for the data in the new system	Concentrate on <i>data requirements</i> only
	2. E-R diagram for the data in the existing system	Differences between this and number one show changes necessary to convert databases to the new application
	3. E-R diagram for all the data in the common database from which the data for the new system will be extracted	Applies if there is a common/shared database in the organisation, which is used by all applications – this is often the case, as the data-oriented approach focuses on providing centralised data stores
	4. E-R diagram for all the data in the common database from which the data in the existing system is extracted	Applies if there is a common/shared database in the organisation, which is used by all applications – this is often the case, as the data-oriented approach focuses on providing centralised data stores

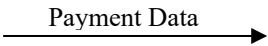
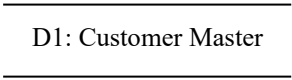
2.1 Process Modelling – Using Data Flow Diagrams (DFDs)

A DFD is a diagram that shows the movement of data between external entities and the processes and data stores within a system.

As can be seen from the deliverables in the table above, the diagrams evolve from the general to the specific – they get more detailed as we move from deliverable number 1 to deliverable number 4.

2.1.1 Symbols

There are various symbols used to represent the parts of a process in a DFD. These are described below. Various writers have used different symbols to represent the components of a system on DFDs – we will use this set of symbols, developed by DeMarco and Yourdon.

Symbol	Meaning
	Data flow – data that are in motion and moving as a unit from one place in a system to another; the line joins two components in the system. The arrow shows the direction of the data flow. A data flow can be between: • Two processes • A data store and a process • An External Entity and a process The text label is the name of the data flow (e.g. Payment).
	Data store – data at rest (a data <i>repository</i>). Processes can enter data into a data store or retrieve data from it. Can represent any physical location for data – e.g. a file folder, computer files, a notebook. The text label is the name of the data store – names should be unique within the system e.g. Customer Master is where Customer data is stored. Each data store is also assigned a number (1 in this case).
	Process – shows what the system does; this is actions performed on data to transform, store or distribute the data. Can be a manual or a computerised process. The text label represents the name of the process; each process should also have a number – the name and number are unique.
	External Entity (also called a <i>source/sink</i>) – supplies data into the system (source) or use output from the system (sink); it can be the origin and/or destination of data, outside the system. We are not concerned with what the external entity does with the data or how it gains access to receive or send data – as it is outside the scope of the system. The text label represents the name of the external entity.

Note: you can read a description of these symbols and how they are used in either *Ref A (page 142)* or *Ref B (page 147)*.

Next, we will look at some examples of DFDs. We will take the Food Ordering System of a restaurant as an example.

2.1.2 Context Diagram.

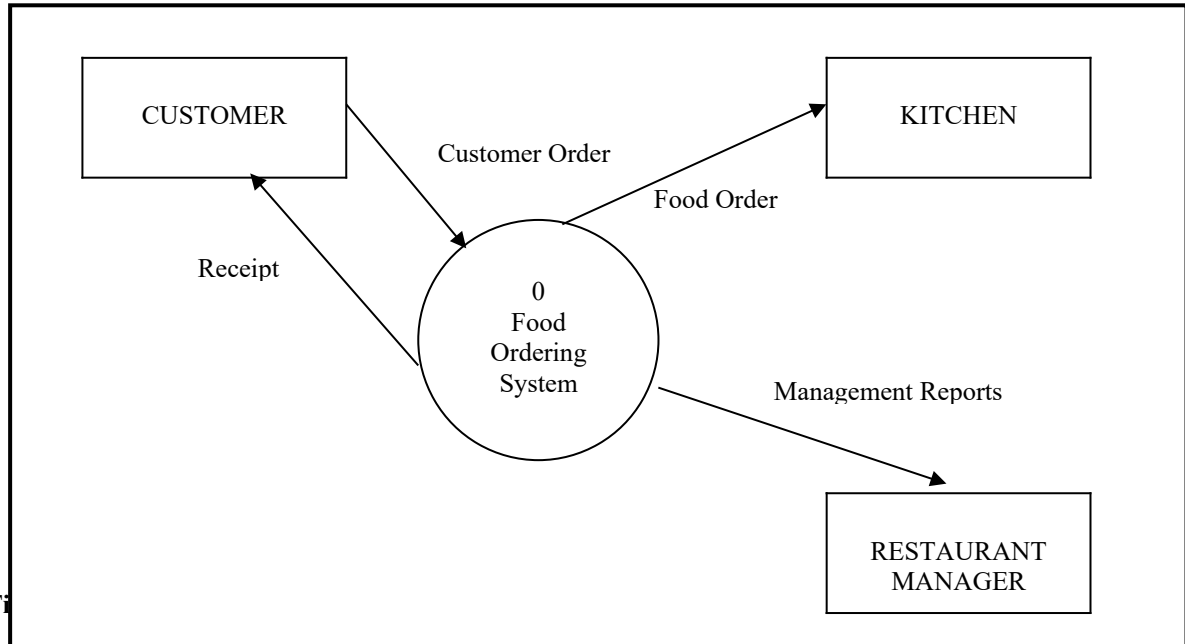
The diagram in Figure 1 below shows a **Context Diagram** for the Food Ordering System of a restaurant. This provides a general overview of the system.

Things to note:

- The Context Diagram shows only one process – this represents the entire system.
- There are three External Entities: _____, _____ and _____. These entities represent the boundary between the system and its environment.
- There are data flows between the External Entities and the process. The data in the system must come from somewhere, and if data goes out of the system, it must go

somewhere. These data flow represent the interfaces between the system and its environment.

- There are no data stores on a Context diagram – this is because the data stores are conceptually *inside* the process – so they are not shown until we start *decomposing* the 'Food Ordering System' process.



Fi

The next step is to consider the context diagram, and think about what processes are inside the system. This will lead to the next diagram – the Level 0 Diagram.

2

T
fo
A
a
T

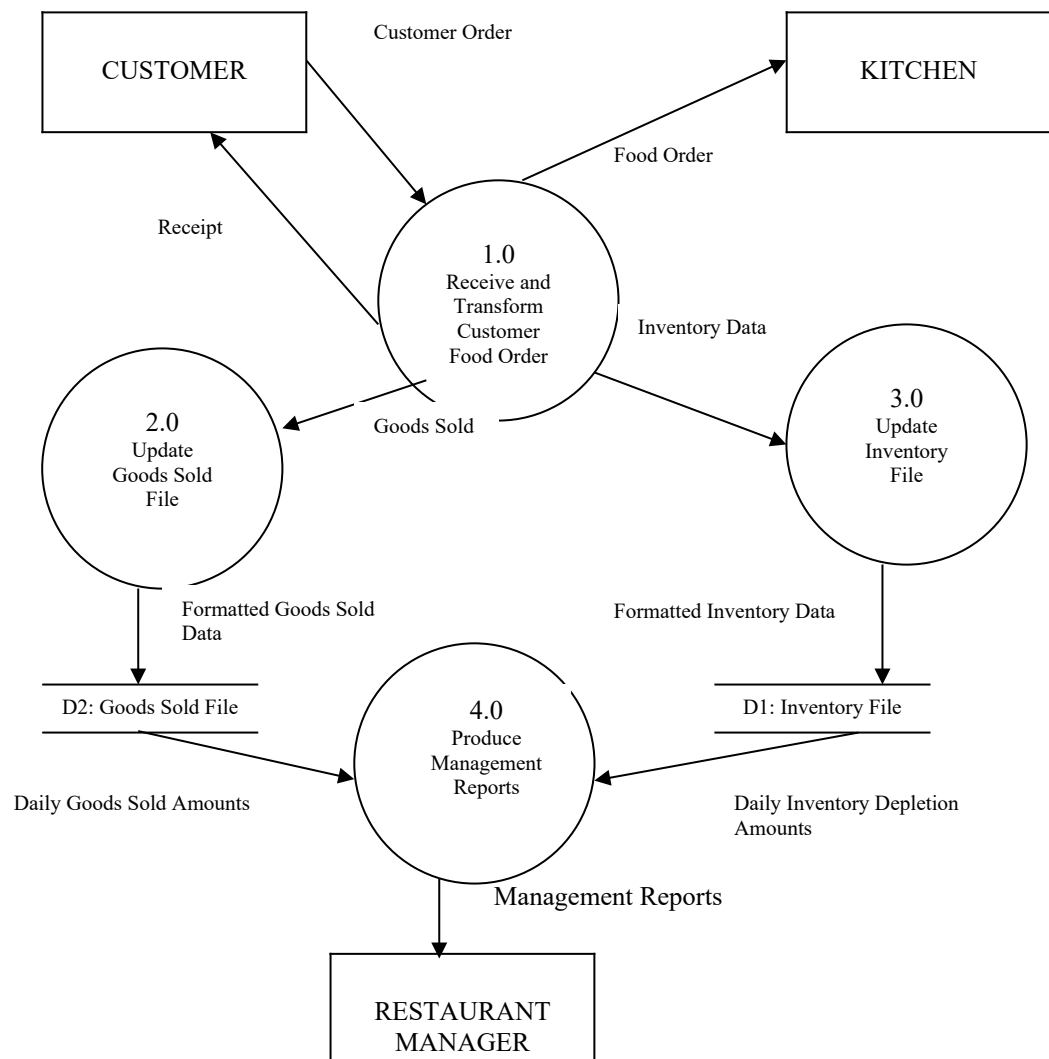


Figure 2 – Level-0 diagram for a restaurant Food Ordering System

Things to note about this diagram:

- It does not show timings e.g. for a data flow, it does not show when or how frequently the data is transferred, or the volume of data that is transferred. The DFD **hides physical characteristics** about data and processes.
- Some processes are **coupled** – this means that when a process, process A, produces a data flow that becomes input to another process, process B, then process B must be ready to accept the data. On the diagram in Figure 2 above, process ____ and process ____ are coupled, also process ____ and process ____.
- Some are **decoupled** – for example, if the data flows into a data store and then into a further process. In this case, the second process can accept the input at any time. Process ____ and process ____ are decoupled in the diagram above; also process ____ and process ____.

2.1.4 Decomposition – Level-n Diagrams

The act of breaking the single system into separate processes is called **functional decomposition**. The above diagram can be further *decomposed*, to break each of the 4 processes into further processes.

This results in a set of *hierarchically related diagrams*, in which one process on a given diagram is explained in greater detail on another diagram. Processes are broken down into *sub-processes*.

Decomposition can continue until no sub-process can be logically broken down any further.

The lowest level of a DFD is called a *primitive DFD*.

This hierarchy is illustrated in Figure 3 below. The levels can continue to level-3, level-4 and so on, until primitive DFDs are reached.

As a general rule, no DFD should have more than about 7 processes in it, as this would make the diagram too crowded and difficult to understand.

Context diagram

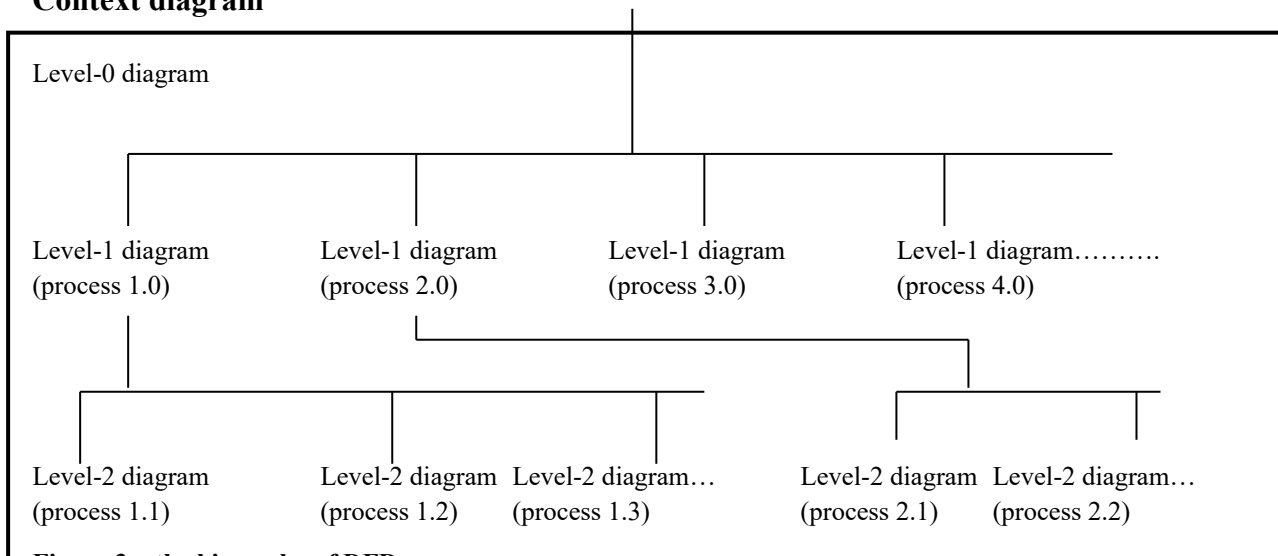


Figure 3 – the hierarchy of DFDs

2.1.5 Rules & Guidelines for Data Flow Diagramming

There are rules and guidelines to follow when drawing DFDs. These rules can help you to produce correct diagrams, and should also help to decide what goes where on the diagrams.

General guidelines – these apply most of the time:

- Objects should have unique names (but can repeat the same object on a diagram, to keep it organised and uncluttered). Be careful with data flows – if two data flows have the same name, they should be *exactly* the same. Even if only one part of the data is different, the data flows should have different names.
- Use meaningful names – the names used for processes, external entities and data flows should indicate what the item is or does.
- The inputs to a process are different from the outputs of that process. This is because a process must have a purpose, so it must transform the input into the output – data does not pass through a process for no reason. The same data can be an input and an output of a process, but the process in this case also produces other new data flows – which are the result of manipulating the input.

Rules

The rules shown in the table below should be followed when drawing DFDs. Some of them are (almost) obvious e.g. a process must get data from somewhere if it produces outputs, so a process cannot have only outputs. Others are not so obvious but do make sense in the context of a system.

Symbol	Rules
Process	- No process can have only outputs – this would mean that it is making data from nothing (a miracle!). If an object has only outputs, then it must be a source/external entity. - No process can have only inputs. If it has only inputs, then it must be a sink/external entity. - A process has a verb phrase label i.e. the name includes a verb, to indicate the action carried out by the process. The verb used is often the same as the verb that might be used in a computer programming language – e.g. print, read, write, sort (this helps with mapping to computer functions later on)
Data Store	- Data cannot move directly from one data store to another – data must be moved by a process - Data cannot move directly to/from an outside sink or source (external entity) from/to a data store. Data must be moved by a process. - A data store has a noun phrase label i.e. the name includes a noun that indicates what is stored in the data store
External Entity	- Data cannot move directly from a source to a sink. If the data are or any concern to the system, they must be moved by a process. Otherwise, the data is not shown on the DFD.
Data Flow	- A data flow has only one direction of flow between symbols. It may flow in both directions between a process and a data store – to show a read before an update. However this is usually shown by two separate arrows – because they happen at different times. - A fork or split in a data flow means that exactly the same data is going from one common location to two or more different processes, data stores or external entities. This usually means that different copies of the data are going to the same locations. - A join in a data flow means that exactly the same data come from two or more different processes, data stores or external entities, to go to a common location. - A data flow cannot go directly back to the same process it leaves. There must be at least one other process that handles the data flow, produces some other data flow and then returns the original data flow to the beginning process (otherwise there is no reason for the data to leave the process in the first place). - A data flow to a data store means that data is being inserted, updated (changed) or deleted - A data flow from a data store means that data is being retrieved or used. - A data flow has a noun phrase label i.e. the name includes a noun that indicates what the data contains. More than one noun phrase label can appear on a single arrow, as long as all of the flows on the same arrow move together as one package.

Balancing DFDs

When *decomposing* a DFD from one level to the next, inputs and outputs to a process need to be conserved in the next level. This is called **balancing**.

Refer back to Figure 1 – the Context Diagram and to Figure 2 – the Level-0 diagram.

In Figure 1, there is one input to the system and three outputs.

In Figure 2, the same one input and the same three outputs are represented. No new inputs or outputs have been introduced.

This means that we can say that the context diagram and the level-0 DFD are *balanced*.

As the system is decomposed, the successive levels of DFD should be balanced. If extra outputs or inputs appear on a level-n diagram, the analyst should review the level-(n-1) diagram and, if necessary, add in the extra data flows to balance the diagrams.

This principle of balancing leads to some more rules that can be applied to data flow diagramming.

Rules to ensure Balanced DFDs
A composite data flow on one level can be split into component data flows at the next level – but no new data can be added and all data in the composite must be accounted for in one or more sub flows. The names of the data flows should indicate what was split.
The input to a process must be sufficient to produce the outputs (including data placed in data stores) from the process. Therefore all outputs can be produced and all data in inputs move somewhere, either to another process or to a data store outside the process or on a more detailed DFD showing a decomposition of that process.
At the lowest level (primitive) DFDs, new data flows may be added to represent data that are transmitted under exceptional conditions. These usually represent error messages from the system (e.g. 'Customer not known; do you want to create a new customer?') or confirmation notices (e.g. 'Do you want to delete this record?')
To avoid data flow lines crossing each other, a data store or external entity can be repeated on a DFD. If doing this, an extra symbol should be used to indicate repeated items. These are shown in Figure 4 below.

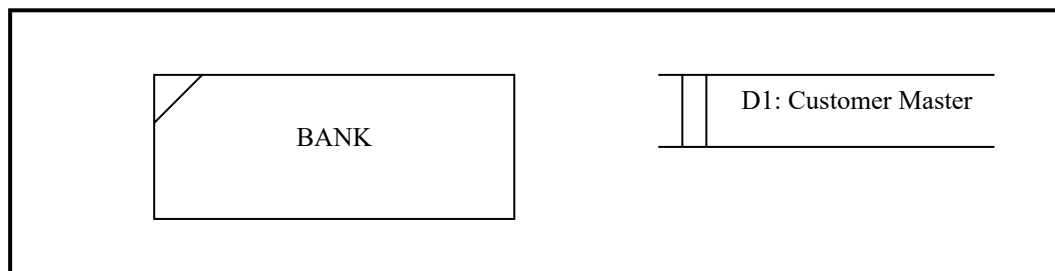


Figure 4 – symbols for repeated External Entity and repeated Data Store

Guidelines

Further guidelines that you should consider when drawing DFDs are as follows.

Completeness – the DFDs should include all of the components necessary for the system being modelled. If a DFD contains data flows that do not go anywhere, or data stores, processes or external entities that are not connected to anything else – then the DFD is *not complete*. If a system requires many DFDs, then it is likely that mistakes will be made. Analysis software or walkthroughs of the DFDs (with other analysts) can help to identify errors.

In addition, all elements of the DFDs should be defined in the project dictionary.

Consistency – the representation of the system at the various levels of DFD should be consistent with each other, from the Context diagram to the primitive DFDs. Some examples of violations of inconsistency:

- a level-1 diagram that has no corresponding level-0 diagram
- a data flow that appears on a higher level DFD but not on lower levels (e.g. on level-1 but not on level-2) – this would be a violation of balancing

- a data flow attached to one object on a lower-level diagram but attached to a different object at a higher level

Timing – DFDs do not show the timings of data flows and other events. Therefore, it is not necessary to consider time when modelling the system using DFDs.

Iterative Development – the process of producing the DFDs is an iterative one – which means that each diagram will normally be drawn a few times before it accurately models the system.

Primitive DFDs – or, when to stop decomposing into further DFDs. Some ways of knowing when to stop are as follows:

- All processes have been reduced to a single decision or calculation or database operation
- When each data store represents data about a single entity (such as a customer, an employee, an order)
- When the system users do not need to see any more details, or when the analysts feel that sufficient detail has been reached
- When every data flow does not need to be split further to show that different data are handled in various ways.
- When it appears that each business form or transaction, computer screen and report has been shown as a single data flow (this often means that each screen and report title corresponds to the name of an individual data flow)
- When it appears that there is a separate process for each choice on all the lowest-level menu options in the system

The primitive DFDs will be very detailed, as they are the lowest level.

DFDs should be used to model the **current physical system** and the **new logical system**.

Usually, the current physical system is modelled first and then the new logical system.

However, some experts believe that analysts should not spend too much time on the current physical system – the higher levels can be done, but the lower, more detailed levels can be omitted. More time and effort should be put into producing the DFDs for the new logical system.

3 Physical & Logical DFDs

When drawing DFDs, the diagrams can model the physical functions of the system and/or the logical functions of the system. This means that there are two types of DFD that can be produced – **physical** and **logical**.

Logical DFDs show *what* happens in a system, without showing *how* it occurs.

Physical DFDs show *how* things happen – by showing the physical components of the system (e.g. the person who carries out a particular job or what computer program is used).

Logical models provide a more clear picture of what the system does, or needs to do, without confusing the picture with details about the current implementation.

However, it is sometimes easier to start with building a physical model of the system, because the physical components are identified during requirements gathering.

The objective of requirements structuring is to produce *logical* DFDs of the current and new systems. Some analysts begin with a physical DFD of the current system, and move from that to a logical DFD of the current system. In some cases, the analyst may begin with a DFD that is a mix of physical and logical. In either case, many of the details of the physical DFD are

not required at later stages – so it is not necessary to spend a lot of time producing physical DFDs.

The logical DFDs are used later in the SDLC, to provide information about the new system and to start building the data model.

4 CASE Tools

CASE stands for Computer Assisted Software Engineering or Computer Aided Software Engineering. There are various CASE applications available to help with all stages of the SDLC. These tools can be useful for requirements structuring as they help to keep track of different models and make it easier to produce detailed models.

Oracle, for example, provides a set of CASE tools for producing and storing models. The tools that support modelling and design are called Oracle Design/2000. There is also a set of tools that support development – these are called Oracle Developer/2000 (for more information see <http://www.oracle.com>).

However, it should be noted that CASE Tools do not *do* the work of the Systems Analyst – they do not create the diagrams and models but are *supporting* tools. CASE Tools generally provide ways of storing models and tracking the different versions and levels of model. They can also feed from one stage of the life cycle into the next e.g. from DFDs to E-R models and from there to logical design.

If you do not have access to CASE Tools, some drawing packages provide limited support for drawing DFDs and other types of system models, by providing the symbols for the diagrams. One example is the Microsoft Visio drawing package.

5 Home Exercise – A Banking System

Consider a bank accounting system. The exercise is to produce DFDs for the personal banking system. The personal banking system is the system that allows customers of the bank, who are individuals, to open accounts, deposit into accounts and withdraw from accounts.

1. Draw the Context Diagram for the Personal Banking System. This should contain at least one external entity (the customer) and more if you think it appropriate. There will also be data flows between the external entities and the process that represents the system.
2. Draw the Level-0 diagram for the Personal Banking System. This diagram will decompose the single process in the Context Diagram into a number of sub-processes. Do not try to break the process down too much – this can be done on the higher level diagrams.
3. Draw the Level-1 diagrams for the processes on the Level-0 diagram for it.

In parts 2 and 3, remember to check that the diagrams are *balanced*.